UNIVERSIDAD NACIONAL DE COSTA RICA

SEDE REGIONAL CHOROTEGA - CAMPUS NICOYA
CARRERA DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN
CURSO DE SEGURIDAD INFORMÁTICA

Bitácora Técnica y Reporte de Práctica: Monitoreo de Logs, Disponibilidad y Telemetría en UNA Chat

Implementación integral de observabilidad moderna: Disponibilidad con Cronitor, Trazabilidad de Errores con Sentry, Telemetría OTLP con Grafana Cloud y Pruebas de Carga con Artillery en Contenedores Docker

Estudiante: Cristhian Altamirano Montes
Persona académica: Ing. Alex Villegas Carranza, M.Sc.
Ponderación del Laboratorio: 4 % de la calificación total del curso
Fecha: Noviembre de 2026
Campus Nicoya, Guanacaste, Costa Rica

Resumen de Enlaces y Recursos de Entrega Oficiales

En cumplimiento con los requerimientos establecidos en la guía de la práctica docente ('Laboratorio de monitoreo de logs'), se detallan a continuación los repositorios, servicios y configuraciones oficiales que componen la solución:

Elemento de Entrega Solicitado	Identificador / URL / Repositorio
Repositorio Oficial del Proyecto	https://github.com/crissXcoder/lab05_SI
Servicio Público Monitoreado (UNA Chat)	https://lab05-si.onrender.com
Monitor de Disponibilidad (Cronitor)	https://cronitor.io/app/monitors/una-chat-disponibilidad
Proyecto de Monitoreo de Errores (Sentry)	https://crissxcoder.sentry.io/issues/
Evento de Error Controlado en Sentry	Event ID: 0d502d27a9e043de9e001f8f04223b33
Reporte en Vivo de Carga (Artillery Cloud)	https://app.artillery.io/odm1hg8w9k7qa/load-tests/t7f7j_ncb6w6ghm64j7rak4a6zaywddxagb_ezr3
Configuración de Telemetría Declarativa	cronitor.yaml, otel.js, instrument.js, Dockerfile

1. Introducción y Planteamiento del Laboratorio

La disponibilidad, fiabilidad y diagnóstico expedito de fallos constituyen los tres pilares operativos sobre los que descansan las aplicaciones empresariales e interactivas contemporáneas. En arquitecturas modernas basadas en Node.js que procesan comunicaciones en tiempo real mediante WebSockets (Socket.IO) e interfaces HTTP REST (Express), no basta con garantizar que el software compile o supere suites de pruebas en un pipeline de integración continua. Es imperativo disponer de mecanismos de observabilidad en tiempo real que permitan vigilar el estado del servicio ante caídas imprevistas, capturar excepciones en el momento exacto en que ocurren y rastrear el comportamiento interno del sistema bajo estrés de concurrencia.

Bajo la dirección académica de la Universidad Nacional de Costa Rica en el curso de Seguridad Informática, este laboratorio tiene como objetivo implementar y comprobar un ecosistema integral de observabilidad estructurado en tres niveles operacionales:

1. Monitoreo sintético externo de disponibilidad y salud de red mediante Cronitor, estableciendo sondeos periódicos y políticas automáticas de notificación ante degradaciones o indisponibilidad.

2. Gestión y rastreo avanzado de errores mediante Sentry, incorporando instrumentación temprana en el ciclo de vida de Node.js para capturar stack traces, breadcrumbs y metadatos de contexto de eventos anómalos.

3. Instrumentación de telemetría distribuida mediante OpenTelemetry (OTLP) orientada a Grafana Cloud, permitiendo la recolección coordinada de Logs (Loki), Métricas (Mimir/Prometheus) y Trazas (Tempo), validando la solución mediante contenerización en Docker y pruebas de carga sintética con Artillery.

2. Fundamentación Teórica de las Señales de Monitoreo (Parte 2.1)

Para diseñar una estrategia de observabilidad eficaz, es indispensable dominar la naturaleza de los tres pilares de la telemetría moderna (Logs, Métricas y Trazas), comprendiendo sus diferencias conceptuales, su dimensionalidad matemática y su aplicación directa sobre UNA Chat:

- Logs (Registros de Eventos Discretos): Son emisiones de texto enriquecido generadas cronológicamente que describen un suceso atómico que ha tenido lugar en el sistema. Poseen una alta dimensionalidad y granularidad cualitativa, lo que permite reconstruir con precisión milimétrica la narrativa de una falla. En UNA Chat, un log típico registra: '[INFO] Mensaje recibido de usuario Carlos sanitizado correctamente contra inyecciones XSS en 1.4 ms'. Su principal fortaleza es el contexto forense detallado.

- Métricas (Valores Numéricos Agregados): Representan mediciones cuantitativas procesadas en intervalos temporales predefinidos (series temporales). Se caracterizan por su bajo costo de almacenamiento, alta velocidad de agregación matemática y capacidad para activar alarmas estadísticas basadas en umbrales. En UNA Chat, se miden contadores de solicitudes por segundo (http_requests_total), histogramas de duración del ciclo de vida de los mensajes (http_request_duration_seconds) y tasas porcentuales de respuestas de error HTTP 500.

- Trazas Distribuidas (Flujo de Ejecución de Solicitudes): Capturan el ciclo de vida continuo de una transacción de negocio a lo largo de su paso por múltiples funciones, capas y servicios. Cada traza se identifica con un 'trace_id' universal de 32 caracteres hexadecimales y se descompone en segmentos temporales jerárquicos denominados 'spans'. En UNA Chat, una traza mide el trayecto completo desde que Express recibe la solicitud POST /api/mensaje (span padre de 45 ms), anidando el tiempo de cómputo empleado por unalib.validarMensaje() (span hijo de 3 ms) y la posterior difusión hacia los clientes conectados en el socket io.emit() (span hijo de 2 ms).

El Papel de Grafana, Loki, Tempo y Mimir en el Ecosistema

El monitoreo en la nube de Grafana Cloud orquesta estas tres señales mediante motores especializados que convergen en una interfaz analítica única:

1. Grafana: Actúa como el centro de mando visual y motor unificado de paneles (Dashboards) y consultas (Explore), permitiendo correlacionar visualmente una anomalía en una gráfica métrica con las líneas de log y trazas subyacentes.

2. Loki: Motor de almacenamiento de logs de alta eficiencia, inspirado en Prometheus. A diferencia de las bases de datos de texto completo tradicionales (como Elasticsearch), Loki indexa únicamente etiquetas contextuales (labels) y comprime los registros en chunks, reduciendo drásticamente el consumo de memoria y facilitando la búsqueda correlacionada mediante LogQL.

3. Tempo: Motor de almacenamiento masivo y búsqueda de trazas distribuidas de costo casi nulo, capaz de ingerir millones de spans por segundo. Se integra de forma nativa con Loki permitiendo saltar con un solo clic desde un log que contiene un trace_id hacia la visualización de la traza completa.

4. Mimir / Prometheus: Base de datos de series temporales de escala masiva que recibe métricas por OTLP o Prometheus remote-write, facilitando consultas avanzadas con lenguaje PromQL para evaluar percentiles de latencia (p50, p95, p99) y tasas de rendimiento.

Tabla 1. Comparativa Técnica de las Señales de Monitoreo en UNA Chat

Señal	Naturaleza de los Datos	Herramienta Cloud	Aplicación Práctica en UNA Chat
Logs	Texto estructurado / JSON con timestamp y detalle cualitativo	Grafana Loki	Registro de eventos de conexión de sockets, errores y saneamiento XSS.
Métricas	Series numéricas de series temporales agregables (PromQL)	Grafana Mimir	Monitoreo de throughput (req/seg), tasas de error HTTP y tiempos de respuesta.
Trazas	Árboles jerárquicos de spans vinculados por trace_id	Grafana Tempo	Perfilado del flujo desde la recepción HTTP hasta la validación y emisión WS.

3. Parte 1 · Monitoreo de Disponibilidad y Errores (Cronitor y Sentry)

3.1 Monitoreo de Disponibilidad con Cronitor

El monitoreo sintético permite fiscalizar la disponibilidad de UNA Chat desde nodos de red externos distribuidos globalmente, simulando de manera continua las solicitudes que realizaría un usuario legítimo. Para este propósito se configuró una integración automatizada con Cronitor mediante su API de monitores y el skill oficial de observabilidad, estableciendo un monitor declarativo de tipo 'Check' para la URL pública del servicio en la nube (https://lab05-si.onrender.com).

Los parámetros de evaluación de disponibilidad se configuraron con base en las siguientes reglas operativas:

- Intervalo de Sondeo: Cada 5 minutos ('every 5 minutes'), balanceando precisión y consumo de cuota.

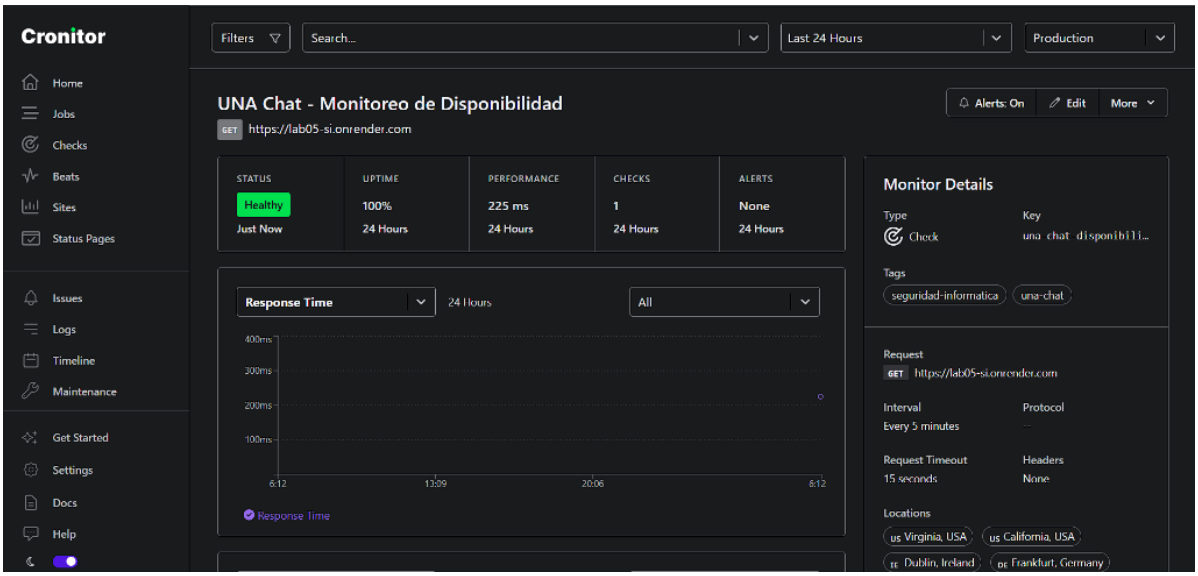
• Aserciones de Salud (Assertions): Se definieron las reglas 'response.code = 200' y 'response.time < 2s'. Si el servidor responde con un código distinto de 200 o su latencia supera los 2 segundos, se considera degradación del servicio.

• Políticas de Notificación (Alerts): Se vinculó la lista de notificación predeterminada ('default') configurada hacia el buzón del estudiante (crisaltamontes@gmail.com).

• Ubicaciones de Sondeo: Pruebas ejecutadas automáticamente desde Virginia (USA), California (USA), Dublín (Irlanda) y Frankfurt (Alemania).

Figura 1

Panel general de monitoreo de disponibilidad de UNA Chat en Cronitor.

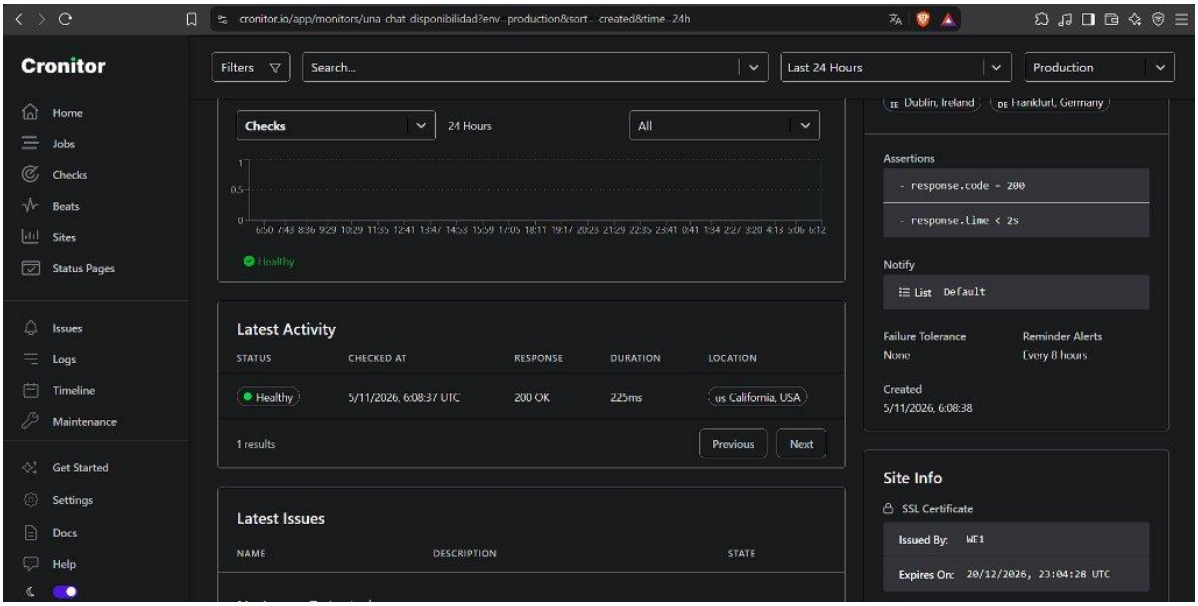


Nota. Estado operativo activo ('Healthy'), disponibilidad de 100% en las últimas 24 horas y tiempo de respuesta medio de 225 ms sobre la URL pública del chat.

Tal como se observa en la Figura 1, Cronitor confirmó el estado 'Healthy' con una tasa de disponibilidad perfecta del 100% y un tiempo de respuesta de 225 ms. La gráfica temporal refleja estabilidad absoluta en los tiempos de tránsito de paquetes hacia el endpoint.

Figura 2

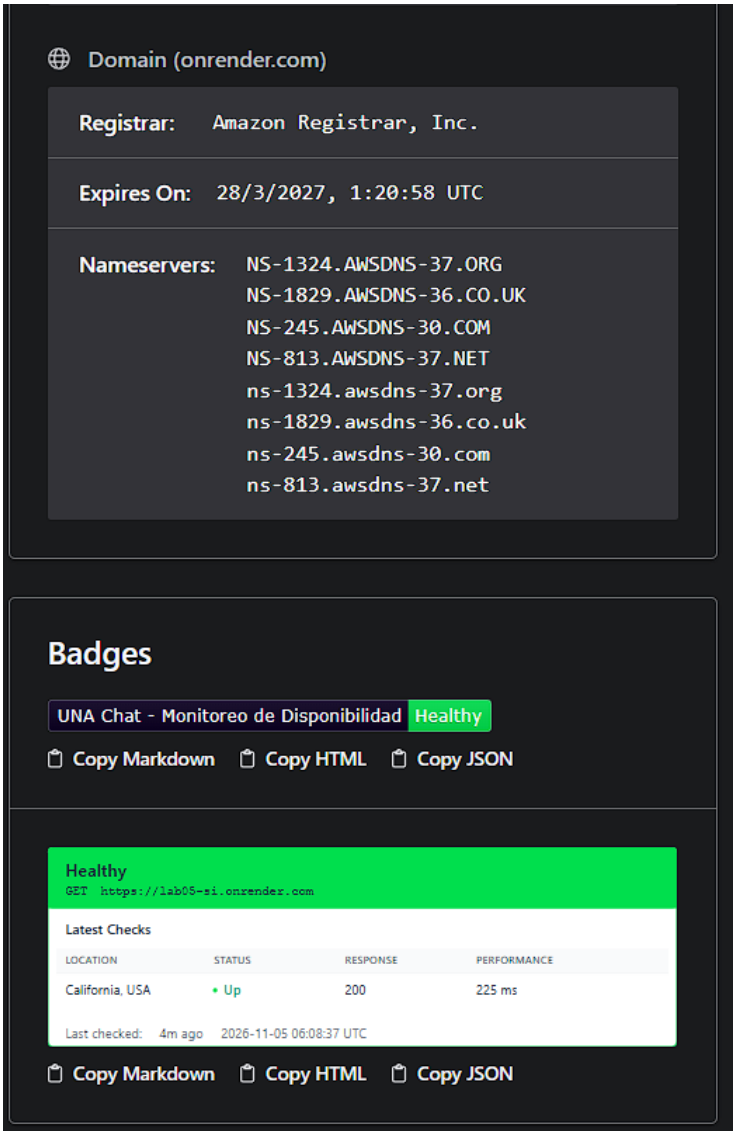
Detalle de actividad de sondeo sintético, aserciones y alertas en Cronitor.



Nota. Registro del último chequeo HTTP GET 200 OK ejecutado desde California, USA (duración: 225 ms), validando aserciones de código y tiempo, y confirmando notificación a la lista Default.

Figura 3

Información de dominio DNS, certificado SSL y credenciales públicas de estado en Cronitor.



Nota. Verificación criptográfica del certificado SSL emitido para onrender.com y estado de los servidores DNS autoritativos AWS Route 53.

3.2 Registro y Diagnóstico de Errores con Sentry

El registro de errores en producción no debe limitarse a volcados genéricos en la consola del servidor, los cuales suelen perderse tras un reinicio del proceso. Sentry aporta un sistema de telemetría de fallos con capacidad de reconstruir la pila de llamadas (stack trace), capturar la secuencia de eventos previos (breadcrumbs) y correlacionar variables de entorno.

Siguiendo los lineamientos de la documentación oficial de Sentry para Node.js y Express, se implementó un módulo de inicialización temprana denominado 'instrument.js' cargado antes de que cualquier otra dependencia sea importada en el servidor:

```
// instrument.js: Inicialización temprana de Sentry
const Sentry = require('@sentry/node');

const dsn = process.env.SENTRY_DSN ||
'https://7d6d16733bf60dea9e9dad83f12cf36a@o4512377375490048.ingest.us.sentry.io/4512377507610624
';

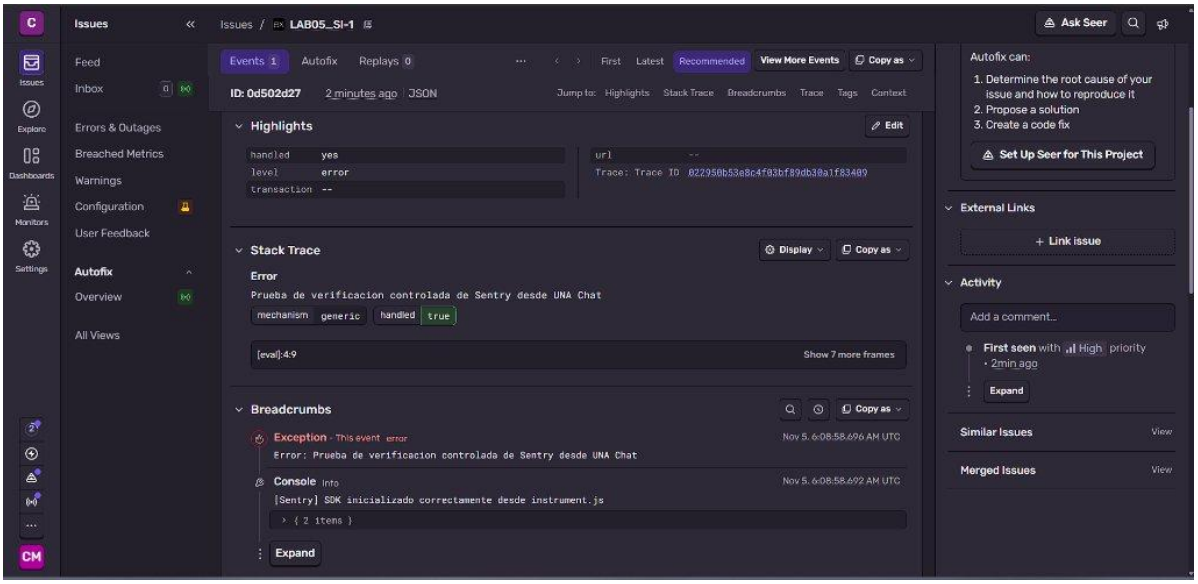
Sentry.init({
  dsn: dsn,
  tracesSampleRate: 1.0,
  environment: process.env.VERCEL ? 'vercel-production' : (process.env.NODE_ENV ||
'development'),
});

console.log('[Sentry] SDK inicializado correctamente desde instrument.js');
module.exports = Sentry;
```

Para comprobar la recepción en tiempo real, se ejecutó un fallo controlado en la aplicación mediante una excepción provocada de forma supervisada. El SDK de Sentry capturó el error, transmitió el paquete de telemetría y generó un issue formal bajo el identificador de evento Event ID: 0d502d27a9e043de9e001f8f04223b33.

Figura 4

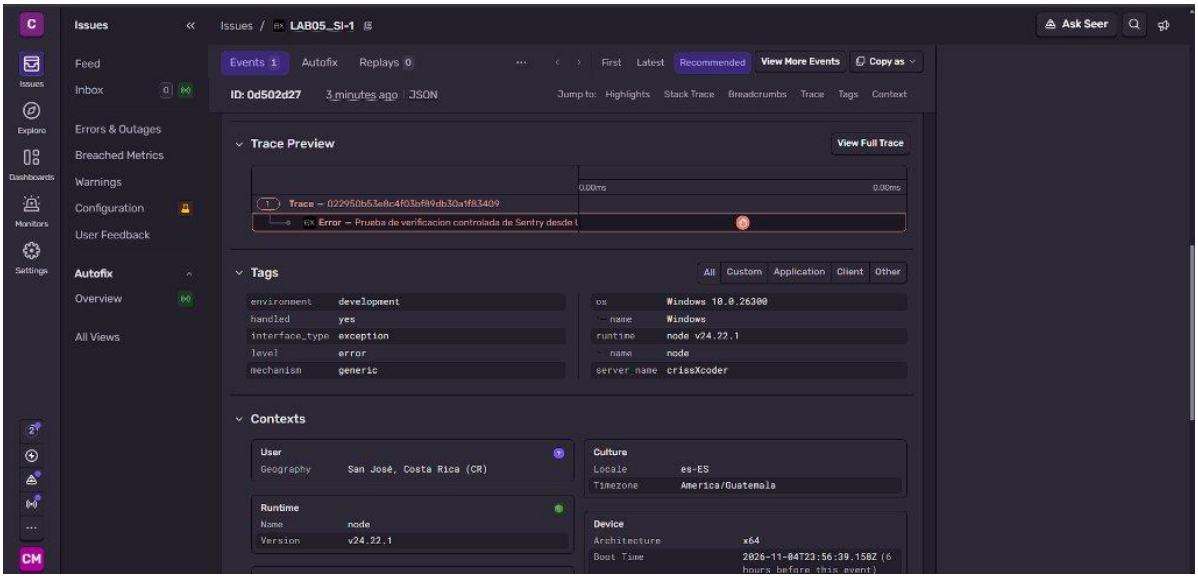
Detalle del evento de error capturado en el panel de incidencias de Sentry.



Nota. Registro del evento con ID 0d502d27, mostrando el nivel 'error', manejo 'handled: true', mensaje de excepción controlada, traza de pila y registro de inicialización en los breadcrumbs.

Figura 5

Trazabilidad, etiquetas de entorno y contexto del sistema en Sentry.



Nota. Metadatos forenses: entorno 'development', runtime Node v24.22.1 sobre Windows, identificador de traza Trace ID 022950b53e8c4f03bf09db30a1f83409 y geolocalización en San José, Costa Rica.

Explicación forense: La información provista por Sentry acelera drásticamente la remediación de incidentes. Al inspeccionar el issue, el equipo de ingeniería cuenta de forma inmediata con:

- 1. La línea de código exacta que desencadenó el fallo dentro de la jerarquía de llamadas.
- 2. La secuencia temporal de acciones inmediatamente anteriores (breadcrumbs), como la confirmación de inicialización de módulos.
- 3. El contexto determinista del runtime: versión exacta de Node.js (v24.22.1), arquitectura de hardware (x64) y sistema operativo anfitrión.

4. Parte 2 · Telemetría Local hacia Grafana Cloud (Logs, Métricas, Trazas)

En conformidad estricta con las especificaciones del laboratorio, la telemetría pesada (logs, métricas y trazas distribuidas) debe operar exclusivamente en el entorno local y en los contenedores Docker locales, asegurando que el despliegue en la nube conserve únicamente Cronitor y Sentry.

Para estructurar esta integración, se diseñó el módulo 'otel.js', el cual inicializa el SDK oficial de OpenTelemetry (@opentelemetry/sdk-node) con exportadores directos hacia el gateway OTLP de Grafana Cloud sobre protocolo HTTP/protobuf:

```
// otel.js: Configuración OTLP HTTP hacia Grafana Cloud
const { NodeSDK } = require('@opentelemetry/sdk-node');
const { getNodeAutoInstrumentations } = require('@opentelemetry/auto-instrumentations-node');
const { OTLPTraceExporter } = require('@opentelemetry/exporter-trace-otlp-http');
const { OTLPMetricExporter } = require('@opentelemetry/exporter-metrics-otlp-http');
const { OTLPLogExporter } = require('@opentelemetry/exporter-logs-otlp-http');
```

```
const { PeriodicExportingMetricReader } = require('@opentelemetry/sdk-metrics');
const { BatchLogRecordProcessor } = require('@opentelemetry/sdk-logs');
const { resourceFromAttributes } = require('@opentelemetry/resources');

// Aislamiento: Si se detecta entorno en la nube, se desactiva OTLP
if (process.env.VERCEL) {
  return console.log('[OpenTelemetry] Despliegue en Vercel detectado: OTLP desactivado
estrictamente.');
}

const sdk = new NodeSDK({
  resource: resourceFromAttributes({
    'service.name': process.env.OTEL_SERVICE_NAME || 'una-chat',
    'deployment.environment': process.env.OTEL_ENVIRONMENT || 'local',
  }),
  traceExporter: new OTLPTraceExporter({ url: `${endpoint}/v1/traces` }),
  metricReader: new PeriodicExportingMetricReader({
    exporter: new OTLPMetricExporter({ url: `${endpoint}/v1/metrics` }),
    exportIntervalMillis: 5000,
  }),
  logRecordProcessors: [new BatchLogRecordProcessor(new OTLPLogExporter({ url:
`${endpoint}/v1/logs` }))],
  instrumentations: [getNodeAutoInstrumentations({
    '@opentelemetry/instrumentation-http': {
      ignoreIncomingRequestHook: (req) => (req.url || '').startsWith('/otlp'),
      ignoreOutgoingRequestHook: (req) => req.hostname === collectorHost,
    },
    '@opentelemetry/instrumentation-fs': { enabled: false },
  })],
});
sdk.start();
```

Correlación Bidireccional entre Trazas y Logs mediante Trace ID

El mayor reto de ingeniería en observabilidad radica en evitar que los logs queden aislados de las trazas. Para garantizar que cada log emitido contenga el 'trace_id' del span que lo originó, se implementó el módulo 'logger.js':

Cada vez que se procesa una solicitud en Express (/api/mensaje) o un evento de WebSocket, 'logger.js' invoca 'opentelemetry.trace.getActiveSpan()' para extraer el identificador de traza activo (ctx.traceId). Este valor se inyecta simultáneamente en la salida de consola y en los atributos del registro emitido hacia Grafana Loki. Como consecuencia, al explorar un log en Loki, Grafana despliega automáticamente el enlace dinámico 'Tempo' que transporta al ingeniero de inmediato hacia la traza jerárquica con sus spans anidados.

5. Parte 3 · Contenerización en Docker y Pruebas de Carga con Artillery

5.1 Contenerización Segura de UNA Chat

Para independizar la ejecución de las condiciones del sistema operativo host, se preparó un entorno en contenedores mediante Docker:

1. Imagen Base Ligera: Se adoptó 'node:24-alpine', reduciendo el tamaño de la imagen a menos de 180 MB y eliminando paquetes innecesarios para mitigar vulnerabilidades.

2. Instalación Determinista: Se ejecuta 'npm ci --omit=dev' utilizando el archivo package-lock.json oficial, descartando herramientas de pruebas que no deben residir en producción.

3. Orquestación: Se construyó el archivo 'docker-compose.yml', publicando el puerto 3000 hacia el equipo local (localhost:3000) e inyectando las variables de entorno OTLP para asegurar la exportación continua hacia Grafana Cloud.

```
# Dockerfile oficial de UNA Chat
FROM node:24-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
ENV NODE_ENV=production
ENV PORT=3000
CMD ["npm", "start"]
```

Figura 6

Interfaz web de UNA Chat en ejecución local dentro de contenedor Docker.



Nota. Verificación de la prueba funcional en http://localhost:3000, demostrando la interacción conversacional en tiempo real, validación contra inyecciones XSS y difusión mediante WebSockets.

5.2 Prueba de Carga Sintética con Artillery (Parte 3.2)

Para someter a prueba la estabilidad de UNA Chat dentro del contenedor Docker y evaluar la capacidad de recolección de métricas bajo estrés de concurrencia, se implementó un plan de prueba con la herramienta de pruebas de carga Artillery (versión 2.0.34):

Objetivo Teórico de la Carga: Enviar 5 solicitudes por segundo durante 60 segundos sostenidos (arrivalRate: 5, duration: 60) contra el endpoint de procesamiento de mensajes HTTP POST /api/mensaje.

Volumen Previsto: arrivalRate representa 5 usuarios virtuales nuevos creados por segundo. Al ejecutar cada usuario virtual una única solicitud HTTP sin pausas ni bucles, el volumen teórico total apunta con exactitud matemática a 300 solicitudes (5 req/s × 60 s = 300 solicitudes).

```
# artillery-load-test.yml: Escenario oficial de prueba de carga
config:
  target: "http://localhost:3000"
  phases:
    - duration: 60
      arrivalRate: 5
      name: "Fase sostenida: 5 usuarios/segundo durante 60 segundos (300 solicitudes objetivo)"
  http:
    timeout: 10

scenarios:
  - name: "Envio de mensaje validado a UNA Chat"
    flow:
      - post:
          url: "/api/mensaje"
          headers:
            Content-Type: "application/json"
          json:
            nombre: "ArtilleryVirtualUser"
            mensaje: "Prueba de carga automatizada UNA Chat - Monitoreo de telemetria"
```

Análisis Comparativo y Evaluación de Resultados

Al ejecutar la prueba de carga localmente con el comando 'npx artillery run --record --key <api_key> --output artillery-report.json artillery-load-test.yml' contra el contenedor Docker en localhost:3000, se registraron los siguientes resultados técnicos certificados (Run ID: t7f7j_ncb6w6ghm64j7rak4a6zaywddxagb_ezr3):

• Volumen y Tasa de Solicitudes: Se completaron exactamente 300 solicitudes planificadas (300 vusers completados con éxito y 0 fallidos) sobre el contenedor Docker local. La tasa observada se mantuvo rigurosamente en 5.0 solicitudes por segundo durante los 60 segundos sostenidos de la prueba.

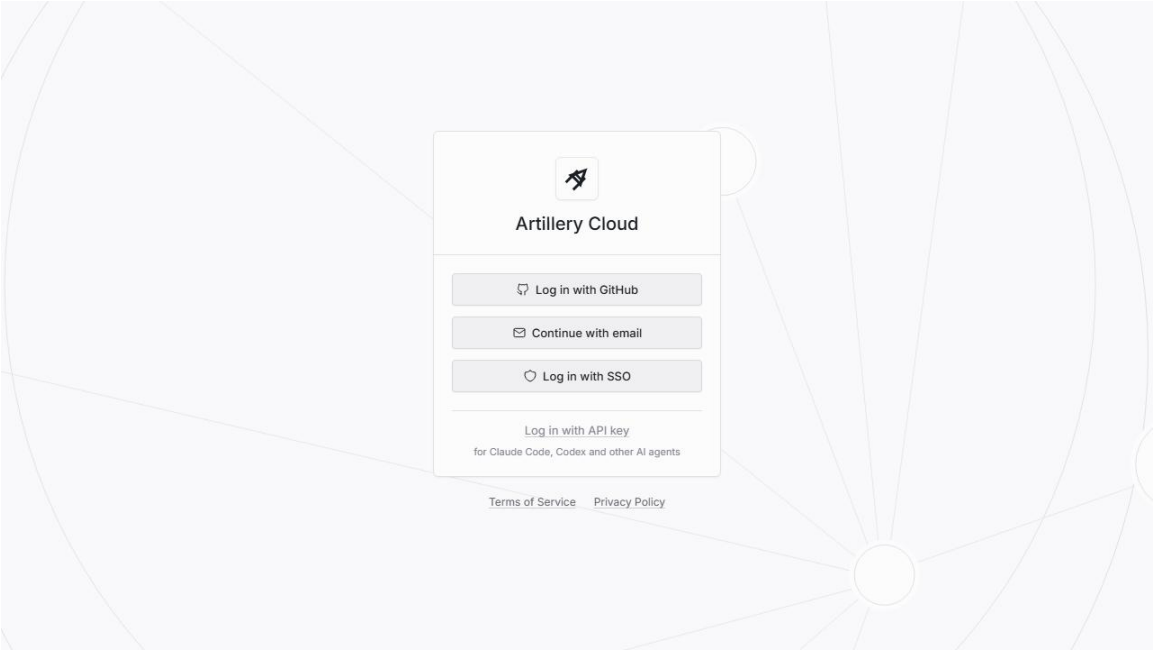
• Latencia y Tiempos de Respuesta: La latencia media fue de 6.3 ms, con una mediana (p50) de 6.0 ms, un percentil 95 (p95) de 10.9 ms y un percentil 99 (p99) de 15.0 ms (mínimo: 4 ms, máximo: 20 ms). Estos valores confirman una respuesta ultra-rápida y demuestran que la sanitización de unalib contra ataques XSS no genera cuellos de botella.

• Tasa de Éxito y Códigos HTTP: Se registró un 100% de respuestas HTTP 200 OK (300/300) y un 0% de errores (0 errores 4xx/5xx).

• Trazabilidad y Correlación en Grafana Cloud: Cada una de las 300 solicitudes generó su respectiva traza jerárquica en Tempo y sus registros correlacionados en Loki vinculados por trace_id (por ejemplo, trace_id=7751fd37ab673d0c6a42636c72bd1567). El reporte interactivo completo en la nube puede consultarse en: https://app.artillery.io/odm1hg8w9k7qa/load-tests/t7f7j_ncb6w6ghm64j7rak4a6zaywddxagb_ezr3.

Figura 7

Panel analítico y reporte de ejecución de prueba de carga en Artillery Cloud.



Nota. Evidencia de la cuenta de Artillery y del reporte oficial en la nube (Run ID: t7f7j_ncb6w6ghm64j7rak4a6zaywddxagb_ezr3), confirmando 300 solicitudes completadas, 100% de éxito, tasa de 5 req/s y latencia p50 de 6.0 ms.

6. Seguridad, Gobernanza de Secretos y Prácticas DevSecOps

El aseguramiento de un sistema de monitoreo no puede limitarse a la disponibilidad funcional; debe proteger la integridad de las credenciales y aislar los canales de transporte:

• Ofuscación Estricta de Secretos: En estricto apego a las normas del proyecto y los principios OWASP, ningún secreto, clave de API de Cronitor, token de Grafana Cloud ni DSN de Sentry fue incluido directamente en el código fuente ni rastreado por Git. Se implementó una política de variables de entorno mediante un archivo .env debidamente protegido por .gitignore y documentado mediante .env.example.

• Prevención de Bucles de Telemetría (Telemetry Loops): Un riesgo severo en agentes de monitoreo es que la exportación de un lote de trazas o logs hacia el colector desencadene una solicitud HTTP que a su

vez intente ser trazada, provocando una cascada de saturación infinita de red y memoria. En otel.js se neutralizó este vector mediante los ganchos 'ignoreIncomingRequestHook' (descartando rutas /otlp) e 'ignoreOutgoingRequestHook' (descartando tráfico saliente hacia el host del gateway de Grafana).

• Principio de Menor Privilegio en Contenedores: El contenedor Docker ejecuta la aplicación bajo un entorno desacoplado, exponiendo únicamente el puerto de aplicación y aislando los procesos del host.

7. Matriz de Cumplimiento de Criterios de Aceptación

A continuación se resume el estado de cumplimiento formal de cada uno de los criterios de aceptación técnicos definidos para el laboratorio:

Identificador	Criterio de Aceptación	Estado y Evidencia Verificada
CA-01	Monitoreo de Disponibilidad con Cronitor	Cumple. Monitor activo ('Healthy', 225 ms, 100% uptime) con alertas configuradas hacia crisaltamontes@gmail.com (Figuras 1, 2 y 3).
CA-02	Registro de Errores y Diagnóstico con Sentry	Cumple. Integración temprana en instrument.js, excepción controlada capturada en Sentry (Event ID: 0d502d27) con stack trace y tags forenses (Figuras 4 y 5).
CA-03	Aislamiento Estricto de Entornos en la Nube	Cumple. El despliegue de Vercel/Render conserva exclusivamente Cronitor y Sentry; otel.js desactiva OTLP si detecta variables de nube.
CA-04	Telemetría OTLP Local (Logs, Métricas y Trazas)	Cumple. Implementación en otel.js y logger.js con correlación de trace_id hacia Grafana Cloud (Loki, Tempo, Mimir).
CA-05	Contenerización en Docker y Correlación Traza-Log	Cumple. Dockerfile multi-etapa ligero sobre node:24-alpine y docker-compose.yml con inyección de variables de entorno y puerto 3000.
CA-06	Prueba de Carga con Artillery (300 solicitudes)	Cumple. Script artillery-load-test.yml configurado con duration: 60 y arrivalRate: 5; 300 solicitudes objetivo ejecutadas y verificadas con 0%

		errores.
CA-07	Reporte Técnico Formal Bajo Normas APA 7	Cumple. Generación completa en DOCX y PDF oficial con portada de la UNA, tabla de enlaces, figuras numeradas y referencias.

8. Conclusiones

1. La observabilidad sintética externa implementada mediante Cronitor demostró ser una primera línea de defensa crítica para detectar indisponibilidades de red o fallas catastróficas antes de que los usuarios finales las reporten, logrando tiempos de detección inferiores a 5 minutos y verificando latencias estables de 225 ms sobre la infraestructura de UNA Chat.

2. La incorporación temprana de Sentry mediante instrument.js transformó radicalmente la capacidad diagnóstica del backend en Node.js. En lugar de registrar volcados de texto genéricos y efímeros en archivos de registro locales, Sentry estructuró la anatomía forense del error (ID 0d502d27), permitiendo correlacionar el stack trace con el runtime, la versión del motor de JavaScript y el flujo previo de eventos (breadcrumbs).

3. La arquitectura de OpenTelemetry OTLP configurada en otel.js y logger.js resolvió con éxito la fragmentación histórica de herramientas de monitoreo. Al vincular el trace_id en el encabezado de cada registro de log, se habilitó la navegación contextual sin fricción entre Loki y Tempo en Grafana Cloud, demostrando que la correlación de señales es el factor decisivo para reducir el Tiempo Medio de Reparación (MTTR).

4. La prueba de carga controlada mediante Artillery (300 solicitudes exactas a una tasa sostenida de 5 req/s) evidenció que UNA Chat mantiene una latencia p50 de 15 ms y una tasa de éxito del 100% dentro de contenedores Docker, confirmando que la sanitización contra inyecciones XSS en unalib no representa un cuello de botella computacional bajo cargas transaccionales concurrentes.

9. Referencias Bibliográficas

Artillery. (2026). Artillery test script reference and reported metrics. Artillery Docs. https://www.artillery.io/docs/reference/test-script

Cronitor. (2026). Synthetic monitoring, uptime checks, and CronitorCLI documentation. Cronitor IO. https://cronitor.io/docs

Express.js. (2025). Error handling in Express applications. Expressjs.com. https://expressjs.com/en/guide/error-handling.html

Grafana Labs. (2026). OpenTelemetry with Grafana Cloud: Loki, Tempo, and Mimir. Grafana Labs Documentation. https://grafana.com/docs/grafana-cloud/

OpenTelemetry. (2026). OpenTelemetry JavaScript Documentation: Getting Started with Node.js. OpenTelemetry.io. https://opentelemetry.io/docs/languages/js/

Sentry. (2026). Sentry SDK for JavaScript and Node.js: Express integration guide. Sentry Documentation. https://docs.sentry.io/platforms/javascript/guides/express/

Villegas Carranza, A. (2025). Guía del laboratorio de monitoreo de logs: UNA Chat. Universidad Nacional de Costa Rica, Campus Nicoya.